

Exploring Methods for PCB Defect Detection and Classification

Project Documentation

Methods Explored

1. Image Subtraction (Classical Computer Vision)
2. Anomaly Detection (WideResNet-50 + FAISS)
3. Binary Classification (EfficientNet-B0)
4. Multi-class Object Detection (YOLOv8s)

Vidisha Agarwal

22B1218

Guide

Prof. Rajbabu Velmurugan

Contents

1	Introduction and Problem Statement	3
1.1	Background and Motivation	3
1.2	Problem Statement	3
1.2.1	Imbalanced and Partially Annotated Dataset	3
1.2.2	No Single Unified Solution	4
2	Dataset Overview	4
2.1	Image Collection	4
2.2	Annotation	4
2.3	Dataset Summary	5
3	Method 1: Image Subtraction	5
3.1	Motivation	5
3.2	Pipeline Overview	6
3.3	Technical Details	6
3.3.1	Anchor-based Alignment	6
3.3.2	ROI Extraction	6
3.3.3	Histogram Matching	7
3.3.4	Image Subtraction	7
3.3.5	SSIM Scoring	8
3.3.6	Morphological Processing and Contour Filtering	8
3.3.7	Verdict Logic	8
3.4	Results	9
3.5	Advantages and Limitations	10
4	Method 2: Anomaly Detection	11
4.1	Motivation	11
4.2	Approach: PatchCore-style Feature Memory Bank	12
4.2.1	Backbone: WideResNet-50	12
4.2.2	Memory Bank Construction	12
4.2.3	Anomaly Scoring	12
4.2.4	Threshold Selection	13
4.3	Evaluation Setup	13
4.4	Results	13
4.4.1	Score Statistics	13
4.4.2	Threshold Sweep	14
4.4.3	Classification Report at Threshold = 0.390	15
4.5	Advantages and Limitations	16

5	Method 3: Binary Classification	16
5.1	Motivation	16
5.2	Model Architecture	16
5.3	Dataset Split and Preprocessing	17
5.4	Handling Class Imbalance	18
5.5	Training Strategy	19
5.6	Results	20
5.6.1	Key Metrics	20
5.6.2	Classification Report	20
5.6.3	Analysis	21
5.7	Advantages and Limitations	21
6	Method 4: YOLOv8 Multi-class Defect Detection	21
6.1	Motivation	22
6.2	Data Preparation	22
6.2.1	COCO to YOLO Format Conversion	22
6.2.2	Dataset Split and Augmentation	22
6.3	Model and Training Configuration	23
6.3.1	YOLOv8s Architecture	23
6.3.2	Training Hyperparameters (Best Run)	24
6.4	Explainability: Grad-CAM	25
6.5	Results	26
6.5.1	Test Set Evaluation	26
6.5.2	Per-class AP@0.5	26
6.5.3	Effect of Confidence Threshold	27
6.5.4	Analysis	27
6.6	Advantages and Limitations	27
7	Comparative Analysis	28
7.1	Output Granularity and Data Requirements	28
7.2	Quantitative Performance	29
7.3	Capability Comparison	30
8	Conclusion and Future Directions	31
8.1	Conclusion	31
8.2	Future Directions	32
8.2.1	Annotate More Images and Increase Dataset Size	32
8.2.2	Hybrid Model Pipelines	33
8.2.3	Semi-supervised Learning with YOLO Pseudo-labels	33

1 Introduction and Problem Statement

1.1 Background and Motivation

Printed Circuit Boards (PCBs) form the backbone of virtually all modern electronic devices. The quality of a PCB directly determines the reliability of the end product, making defect detection an essential step in the manufacturing process. PCB fabrication involves substrate preparation, copper trace etching, solder mask application, component placement, and reflow soldering — each step introducing potential failure modes such as misplaced components, cold solder joints, solder bridges, and physical damage.

Traditionally, quality control relies on **manual visual inspection**, where trained technicians examine boards under magnification and flag anomalies. While effective at small scale, manual inspection is slow, subjective, prone to fatigue-related errors, and does not scale to modern high-volume production. These limitations motivate the development of **automated visual inspection** systems using computer vision and machine learning.

1.2 Problem Statement

This project addresses **automated defect detection and classification in PCB component images**. Given such an image, the system should produce one or more of the following outputs depending on the method:

1. A binary decision: *defective* or *good*.
2. The spatial location of the defect as a bounding box.
3. The type of defect, from a set of 15 predefined categories.

The problem is challenging due to the following factors.

1.2.1 Imbalanced and Partially Annotated Dataset

The dataset comprises **1000+ good PCB images** and **207 defective images**, giving a ratio exceeding 5:1 in favour of the non-defective class. Of the 207 defective images, only **105 carry CVAT bounding-box annotations** in COCO JSON format. The remaining **102 defective images have no labels** — their defective nature is known but no bounding boxes or class identifiers exist. This directly rules out fully supervised methods for these images and motivates label-free approaches. Within the annotated set, the 15 defect classes are themselves highly imbalanced, with counts ranging from 24 (component solder short) down to 2 (component shift) — well below the practical minimum for reliable per-class supervised learning.

1.2.2 No Single Unified Solution

No single method can simultaneously satisfy all requirements given these constraints. The tension between label requirements, output granularity, and generalisation capability is summarised as follows:

- **Image subtraction** requires no training and no labels, and provides interpretable localisation, but depends on a good reference image and only works for specific positional defects.
- **Anomaly detection** requires no defect labels at all and generalises to unseen defect types, but cannot identify the defect type.
- **Binary classification** handles imbalance well and achieves strong AUC, but provides no localisation and no defect-type output.
- **YOLOv8** produces bounding boxes with class labels in a single pass — the most informative output — but is constrained by the small annotated dataset.

This motivates the exploration of all four approaches as complementary methods.

2 Dataset Overview

2.1 Image Collection

The dataset consists of zoomed-in crops of PCB component regions collected from real PCB samples, divided into two categories:

Good PCB images: Over 1000 images of non-defective component regions, used as the reference distribution for anomaly detection and as the negative class for supervised methods.

Defective PCB images: 207 images containing one or more defects, further divided based on annotation availability as described below.

2.2 Annotation

Defective images were annotated using **CVAT** (Computer Vision Annotation Tool), a widely used open-source annotation platform. Annotators drew bounding boxes around each defect region and assigned one of 15 defect category labels. Annotations were exported in **COCO JSON format**, which stores image metadata, bounding box coordinates in $[x, y, \text{width}, \text{height}]$ form, and category information.

2.3 Dataset Summary

Table 1 summarises the composition of the full dataset.

Table 1: Dataset composition summary.

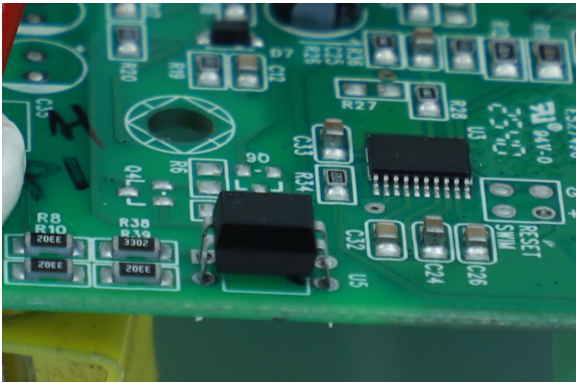
Subset	Count
Good PCB images	1000+
Defective images — CVAT annotated	105
Defective images — unannotated	102
Total defective images	207

3 Method 1: Image Subtraction

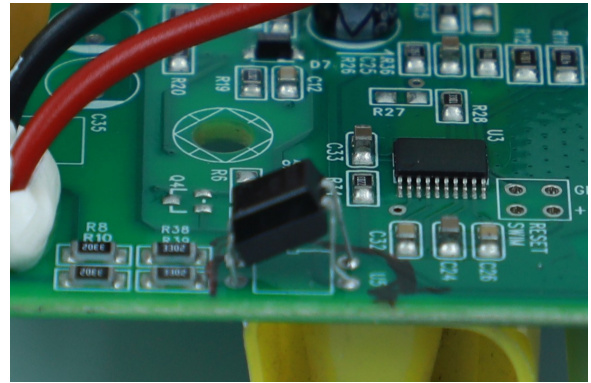
3.1 Motivation

Solder spread and component liftup are **positional defects** — the PCB layout is fixed and repeatable, meaning a defective image and a good reference image of the same component region should be pixel-wise identical except at the defect location. This makes direct pixel-level comparison a natural and interpretable detection strategy. No training data, annotations, or GPU resources are required, making this method ideal as a lightweight first-pass detector for these specific defect types.

Figure 1 shows an example of a pair of good and bad image (liftup).



(a) Non-defective Image



(b) Defective Image- Component Liftup

Figure 1: Example 1 of Image Subtraction Method

3.2 Pipeline Overview

The detection pipeline takes a pair of images — a known-good reference and a test image — and processes them through the following stages:

1. **Anchor-based alignment** via template matching
2. **ROI extraction** using the computed pixel offset
3. **Histogram matching** to normalise lighting differences
4. **Image subtraction** to produce a difference map
5. **SSIM computation** on the aligned patches
6. **Morphological processing** to clean the difference map
7. **Contour filtering** to identify defect regions
8. **Verdict** based on SSIM score and contour area

3.3 Technical Details

3.3.1 Anchor-based Alignment

Small camera or placement variations cause false pixel differences. A stable **anchor component** visible in both images is used to compute the pixel shift $(\Delta x, \Delta y)$ via template matching (`cv2.matchTemplate`, `TM_CCOEFF_NORMED`):

$$R(x, y) = \frac{\sum_{x', y'} [T(x', y') \cdot I(x + x', y + y')]}{\sqrt{\sum_{x', y'} T(x', y')^2 \cdot \sum_{x', y'} I(x + x', y + y')^2}} \quad (1)$$

where T is the anchor template from the good image and I is the test image. The location of maximum R gives the anchor position; the offset between images is applied to extract aligned patches. Anchor details per defect type:

- **Liftup:** 20-pin IC (U3), $Y \in [250, 380]$, $X \in [740, 930]$.
- **Solder spread:** Axial resistor with coloured bands, $Y \in [10, 80]$, $X \in [50, 380]$.

3.3.2 ROI Extraction

Once the offset is computed, the region of interest is extracted differently for each defect type:

- **Liftup:** A centred patch of size 260×260 pixels (`COMP_PAD` = 130 px each side) around the component centre, extracted from both images after applying the offset.

- **Solder spread:** An explicit rectangular ROI $X \in [130, 480]$, $Y \in [80, 320]$, shifted by $(\Delta x, \Delta y)$.

Figure 2 shows the anchor and ROI alignment for Example 1.

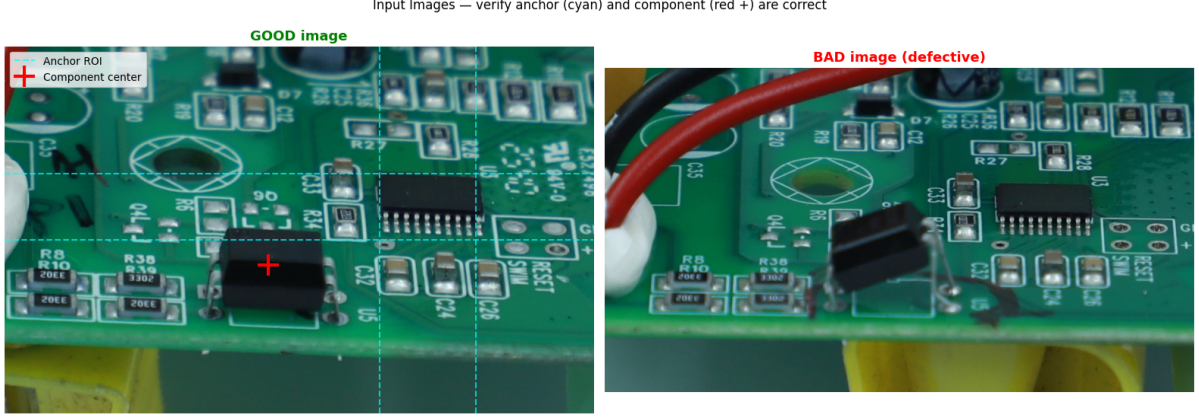


Figure 2: Example 1: Anchor and ROI Alignment

3.3.3 Histogram Matching

Before subtraction, brightness and contrast differences are normalised using **per-channel histogram matching**. The cumulative distribution function (CDF) of pixel intensities in the reference image is computed for each channel:

$$\text{CDF}_{\text{ref}}(v) = \sum_{u=0}^v h_{\text{ref}}(u) \quad (2)$$

A lookup table maps each intensity value in the test image to the value in the reference image with the same CDF value, transferring the reference intensity distribution to the test image and removing lighting-induced false differences.

3.3.4 Image Subtraction

The subtraction strategy differs based on the physical nature of each defect:

Component liftup — absolute difference:

$$D(x, y) = |I_{\text{good}}(x, y) - I_{\text{bad}}(x, y)| \quad (3)$$

A lifted component causes pixel changes in *both* directions (brighter and darker), so absolute difference is used to capture all changes.

Solder spread — directional difference:

$$D(x, y) = \max(I_{\text{bad}}(x, y) - I_{\text{good}}(x, y), 0) \quad (4)$$

Solder spread only *adds* new bright material. Directional subtraction (saturating at zero) specifically isolates pixels that became brighter, suppressing darker noise regions and reducing false positives.

3.3.5 SSIM Scoring

The Structural Similarity Index (SSIM) is computed on the aligned grayscale patches to produce a single scalar quality score:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)} \quad (5)$$

where μ_x , μ_y are local means, σ_x^2 , σ_y^2 are local variances, σ_{xy} is the cross-covariance, and c_1 , c_2 are stabilisation constants. SSIM ranges from 0 (completely dissimilar) to 1 (identical). Unlike pixel-wise difference, SSIM accounts for luminance, contrast, and structural information simultaneously.

3.3.6 Morphological Processing and Contour Filtering

The binary thresholded difference map contains noise pixels and fragmented defect regions. The following steps clean and consolidate the map:

1. **Binary threshold**
2. **Morphological dilation**
3. **Contour detection**

3.3.7 Verdict Logic

The final decision for both defect types follows the same OR rule:

$$\text{Verdict} = \begin{cases} \text{DEFECTIVE} & \text{if SSIM} < \theta \text{ or } |\text{defect contours}| > 0 \\ \text{GOOD} & \text{otherwise} \end{cases} \quad (6)$$

where $\theta = 0.55$ for component liftup and $\theta = 0.50$ for solder spread. Using OR ensures that structural changes caught by SSIM but not by contours (and vice versa) are both detected.

A **Grad-JET heatmap** is generated by applying `cv2.COLORMAP_JET` to the difference map and blending it with the test image at weights 0.4 (original) +0.6 (heatmap), providing a visual localisation of the detected difference. Figure 3 shows the result for Example 1.

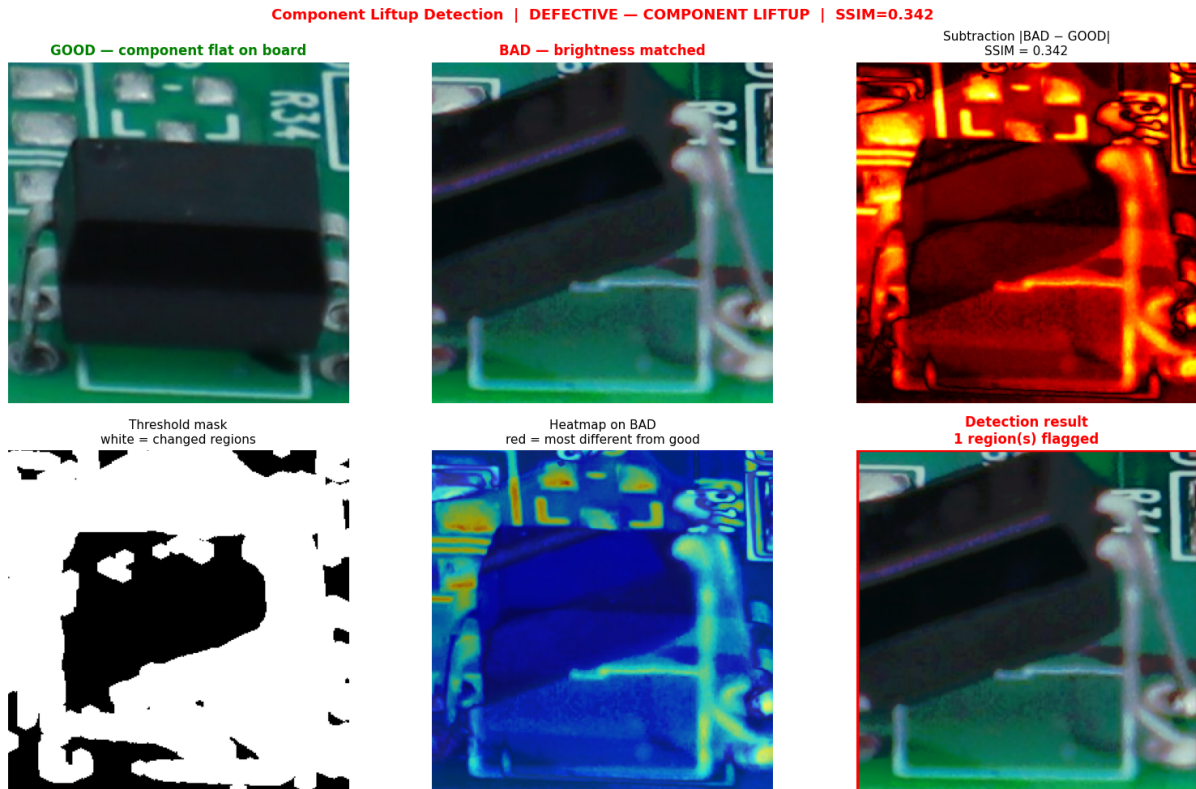


Figure 3: Example 1: Result and JET Heatmap

3.4 Results

The method successfully detects solder spread and component liftup defects when a well-aligned reference image is available. The JET heatmap provides clear visual localisation of the difference region, and the two-stage contour filter effectively suppresses false alarms from normal pad solder blobs. Anchor match confidence above the threshold ensures reliable alignment in the majority of test cases. Figures 4, 5 and 6 show another example of this method applied on solder spread defect.

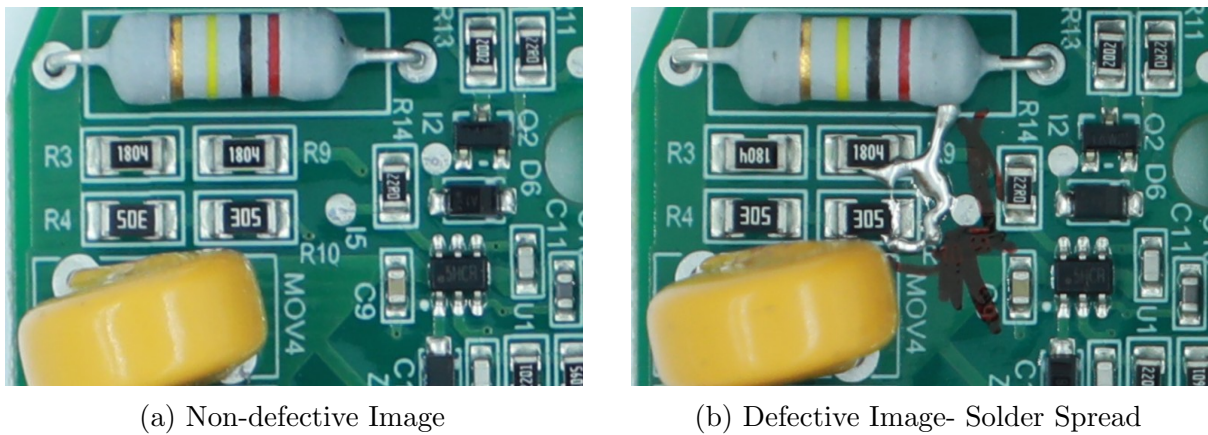


Figure 4: Example 2 of Image Subtraction Method

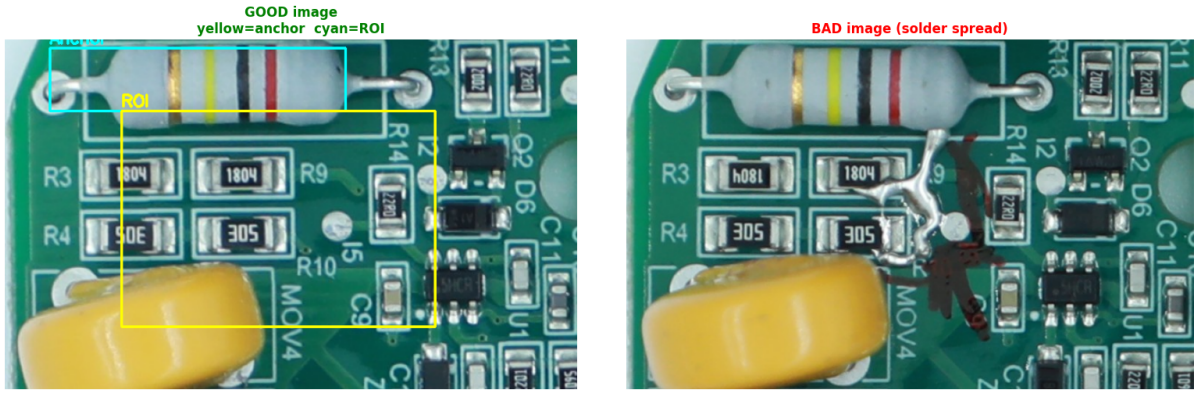


Figure 5: Example 2: Anchor and ROI Alignment

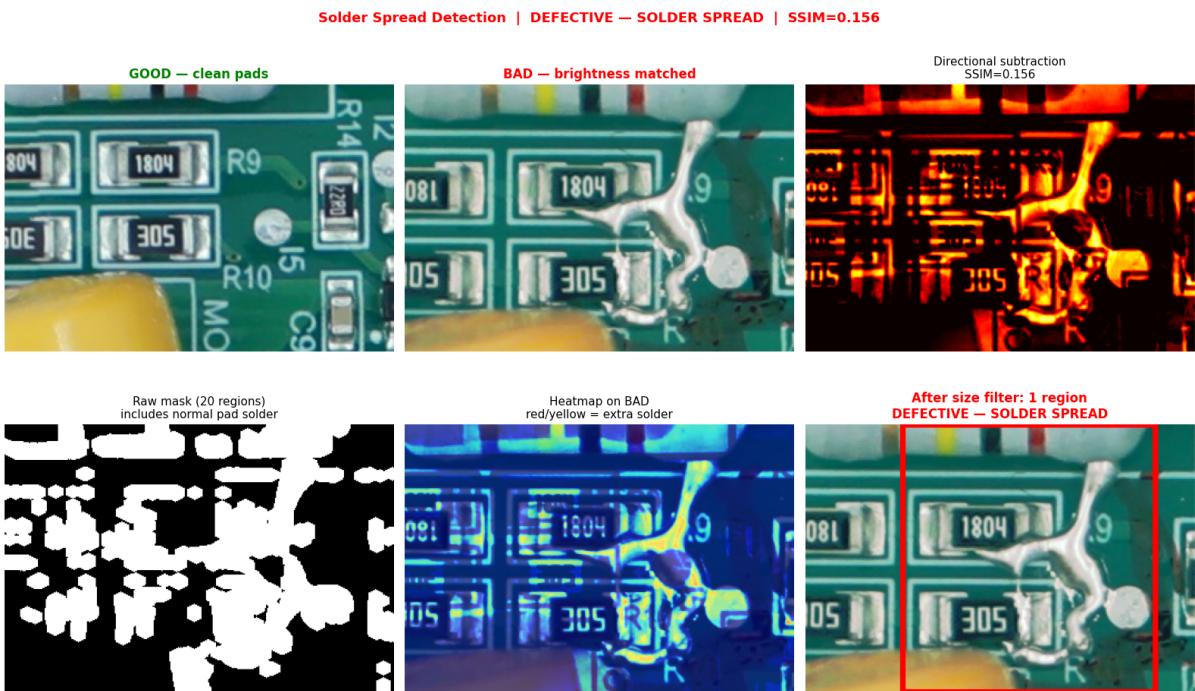


Figure 6: Example 2: Result and JET Heatmap

3.5 Advantages and Limitations

Advantages:

- Requires no training data, annotations, or GPU resources.
- Fully interpretable — the heatmap shows the exact pixel-level location of the detected difference.
- Handles lighting variation between image pairs via histogram matching.
- Fast inference using pure OpenCV operations.
- No defect labels required; only a single good reference image is needed.

Limitations:

- Requires a good reference image for every test case — not always available in production.
- Anchor coordinates and ROI regions are hardcoded per PCB variant and do not generalise automatically to new board designs.
- Cannot classify the defect type — only detects the presence of a difference.
- All threshold and area parameters require manual calibration per defect type and PCB variant.

4 Method 2: Anomaly Detection

4.1 Motivation

102 of the 207 defective images carry no annotations, and the variety of defect types across the full dataset is large. A method that requires defect labels for training is therefore inapplicable to a significant portion of the data. Anomaly detection addresses this by learning only from **good PCB images** — building a representation of what a normal component region looks like — and flagging any test image whose features deviate significantly from this normal distribution. No defect labels are required at any stage, and the method generalises naturally to defect types not seen during setup.

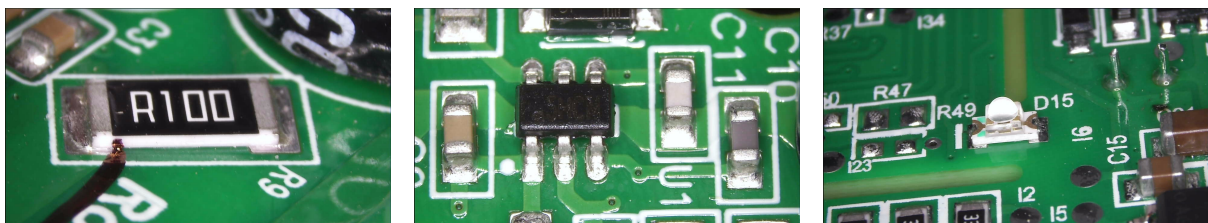


Figure 7: Good PCB Image Samples

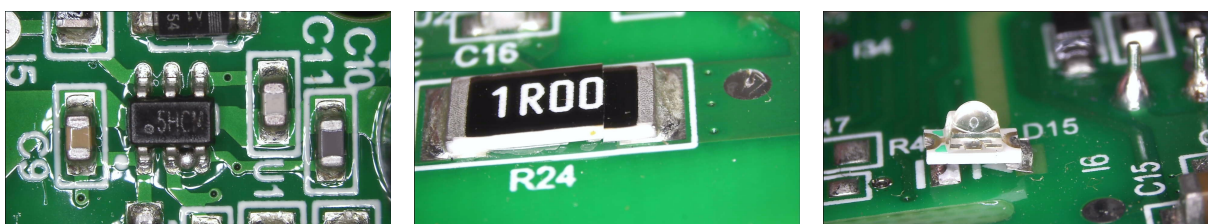


Figure 8: Defective PCB Image Samples

4.2 Approach: PatchCore-style Feature Memory Bank

The approach follows the **PatchCore** paradigm: extract deep features from all good images using a pretrained backbone, store them in a **memory bank**, and at inference time compute the distance from a test image’s features to the nearest entry in the memory bank. A large distance indicates the test image is anomalous.

4.2.1 Backbone: WideResNet-50

Feature extraction uses **WideResNet-50** pretrained on ImageNet. WideResNet is a variant of ResNet where layers are made wider (more feature map channels) rather than deeper, yielding richer representations per layer. The final fully-connected classification head is removed, retaining only the convolutional feature extractor:

```

1 backbone = torchvision.models.wide_resnet50_2(pretrained=True)
2 backbone = torch.nn.Sequential(*list(backbone.children())[:-1])
3 backbone.eval()

```

Listing 1: Backbone setup.

This produces a feature vector of dimension **2048** per image. The backbone is used in inference mode only — no fine-tuning is performed.

Each image is preprocessed to size 224×224 and normalised with ImageNet statistics:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}, \quad \mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225] \quad (7)$$

4.2.2 Memory Bank Construction

All 1061 good PCB patches are passed through the backbone to extract their 2048-dimensional feature vectors. Each vector is then **L2-normalised** to project it onto the unit hypersphere:

$$\hat{f} = \frac{f}{\|f\|_2} \quad (8)$$

The normalised vectors are stacked into a feature matrix of shape (1061, 2048) and indexed using a **FAISS IndexFlatL2** index for efficient nearest-neighbour search. The full memory bank is retained (SUBSAMPLE_RATIO = 1.0), resulting in a memory footprint of **8.7 MB**.

4.2.3 Anomaly Scoring

At inference time, a test image is passed through the same preprocessing and backbone pipeline to produce its normalised feature vector. The anomaly score is defined as the **Euclidean distance** to the nearest neighbour in the memory bank:

$$s(x) = \min_{f_i \in \mathcal{M}} \|\hat{f}(x) - f_i\|_2 \quad (9)$$

where $\mathcal{M}$ is the set of memory bank vectors. Since all vectors are L2-normalised, this is equivalent to cosine distance on the unit sphere.

An important implementation detail concerns the choice of k in the k -nearest-neighbour search:

- **Good patches** (present in the index): $k = 2$ is used and the *second* nearest neighbour is taken (`score` = $\sqrt{D[:, 1]}$), since the nearest neighbour at index 0 is the vector itself (distance ≈ 0).
- **Defective patches** (not in the index): $k = 1$ is used and the nearest neighbour at index 0 is taken (`score` = $\sqrt{D[:, 0]}$), as there is no self-match to skip.

4.2.4 Threshold Selection

Two threshold strategies are supported:

Manual: A fixed threshold of $\theta = 0.39$ is applied directly.

Percentile sweep: The p -th percentile of good patch scores is used as the threshold, and a sweep over $p \in \{80, 85, 90, 95, 99\}$ is performed. The threshold maximising the F1 score on the combined evaluation set is selected.

The verdict is:

$$\text{Verdict} = \begin{cases} \text{DEFECTIVE} & \text{if } s(x) > \theta \\ \text{GOOD} & \text{otherwise} \end{cases} \quad (10)$$

4.3 Evaluation Setup

The evaluation set is constructed by combining:

- All 207 defective image scores (label = 1).
- A random sample of $\min(207 \times 10, 1061) = 1061$ good image scores (label = 0).

This gives **1268 total samples**. Three evaluation outputs are generated: ROC curve with AUC score, Precision-Recall curve with Average Precision (AP), and a confusion matrix with classification report at the chosen threshold.

4.4 Results

4.4.1 Score Statistics

Table 2 shows the anomaly score statistics for both groups.

Table 2: Anomaly score statistics for good and defective patches.

Group	Mean	Std	Min / Max
Good patches (n = 1061)	0.2655	0.1510	— / 1.0711
Defective patches (n = 207)	0.4834	0.1335	0.0000 / —

The **separation** between the two distributions is:

$$\Delta = \mu_{\text{defect}} - \mu_{\text{good}} = 0.4834 - 0.2655 = 0.2180 \quad (11)$$

A positive separation confirms that defective patches consistently score higher than good patches on average, validating the approach. Figure 9 shows the distribution of anomaly scores of both types of images.

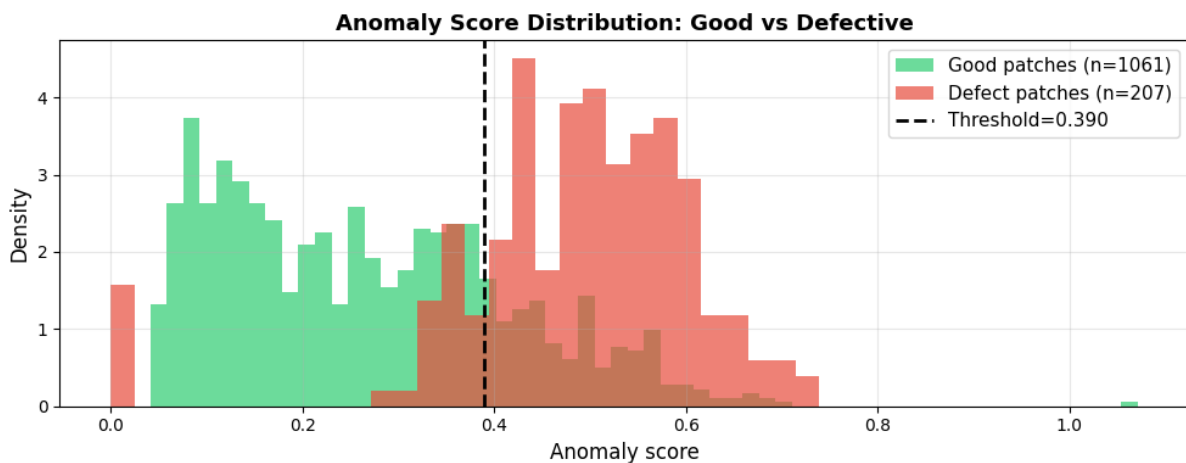


Figure 9: Anomaly Scoring for Good and Defective Images

4.4.2 Threshold Sweep

Table 3 shows results across the percentile sweep.

Table 3: Threshold sweep results (percentile strategy).

Percentile	Threshold	Good FPR	Defect TPR	F1
80 ★	0.3915	19.89%	83.09%	0.5811
85	0.4313	14.99%	72.46%	0.5792
90	0.4866	9.99%	54.59%	0.5293
95	0.5439	4.90%	33.82%	0.4230
99	0.6254	1.04%	8.70%	0.1525

The best F1 of **0.5811** is achieved at the 80th percentile (threshold = 0.3915), which was put in manually and evaluated again.

4.4.3 Classification Report at Threshold = 0.390

Table 4: Classification report at threshold = 0.390 (1268 evaluation samples).

Class	Precision	Recall	F1-Score	Support
Good	0.96	0.80	0.87	1061
Defective	0.44	0.84	0.58	207

The overall **AUC-ROC = 0.8620** and **Average Precision = 0.5236**. The high defect recall (0.84) means the model catches most defective images, at the cost of lower precision (0.44) — a reasonable trade-off for inspection systems where missing a defect is more costly than a false alarm. The overall accuracy is 80%. Figure 10 shows the AUC-ROC and Precision-Recall Curves as well as Confusion Matrix for this method.

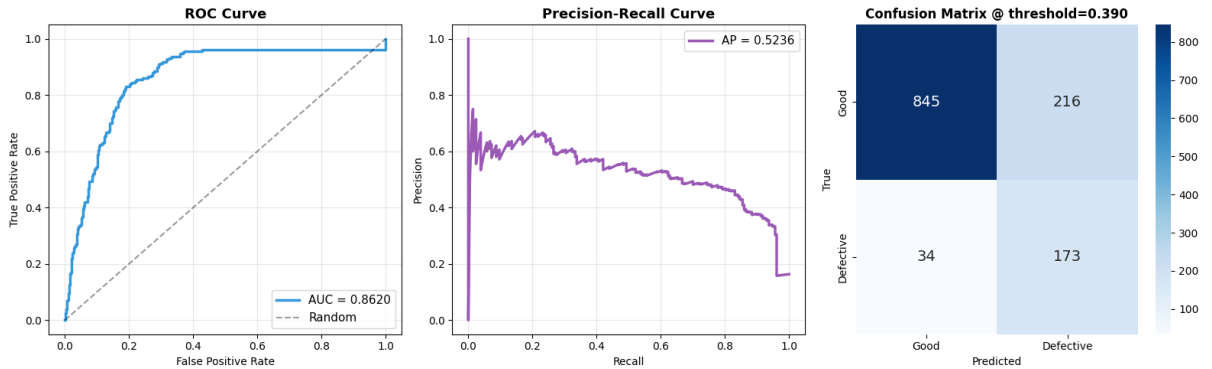


Figure 10: AUC-ROC Curve, PR Curve and Confusion Matrix

4.5 Advantages and Limitations

Advantages:

- No defect labels required — trained entirely on good PCB images.
- Generalises to unseen defect types not present during memory bank construction.
- No fine-tuning of the backbone — ImageNet features transfer effectively without additional training.
- Flexible threshold selection via both manual and percentile-based strategies, with a built-in sweep for optimisation.

Limitations:

- Cannot identify the defect type — only flags an image as anomalous.
- Provides no spatial localisation of the defect within the image.
- Threshold selection requires calibration; an incorrect threshold leads to either high false positive rate or low defect recall.
- Score distributions overlap ($\text{std} \approx 0.15$ for good patches), limiting the maximum achievable F1 at any single threshold.
- Memory and search time grow linearly with the number of good patches in the memory bank.

5 Method 3: Binary Classification

5.1 Motivation

When defect type identification is not required, a supervised binary classifier — predicting *good* or *defective* — offers a simpler and more data-efficient approach than full object detection. Transfer learning from a pretrained convolutional neural network allows strong performance even on small datasets by leveraging features learned from large-scale image corpora. **EfficientNet-B0** was selected for this task due to its strong accuracy-to-parameter ratio, making it well-suited for fine-tuning on limited data while minimising overfitting risk.

5.2 Model Architecture

EfficientNet-B0 is a convolutional neural network that scales depth, width, and input resolution jointly using a compound scaling coefficient, achieving competitive accuracy

with significantly fewer parameters than architectures such as ResNet-50 or VGG. It employs depthwise separable convolutions and Squeeze-and-Excitation blocks, which learn channel-wise feature recalibration.

The original EfficientNet-B0 classification head (a single linear layer mapping 1280 features to 1000 ImageNet classes) is replaced entirely with a custom two-layer head:

$$\text{Head: Dropout}(0.4) \rightarrow \text{Linear}(1280 \rightarrow 256) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3) \rightarrow \text{Linear}(256 \rightarrow 2) \quad (12)$$

The output is a 2-dimensional logit vector, one per class (*defective*, *good*). Softmax probabilities are computed at inference time, and the model is evaluated using the defective class probability $p_{\text{defective}}$.

Total trainable parameters in the head: **328,450**.

5.3 Dataset Split and Preprocessing

The full dataset of good and defective images is split into training and validation sets using an 80/20 stratified split (`RANDOM_SEED= 42`, `VAL_SPLIT= 0.20`):

Table 5: Dataset split for binary classification.

Subset	Good	Defective
Train	849	166
Val	212	41
Total	1061	207

The class ratio in training is approximately 5:1 (good:defective). All images are resized to 224×224 and normalised with ImageNet statistics $\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$. A `RandomHorizontalFlip` augmentation is applied during training only.

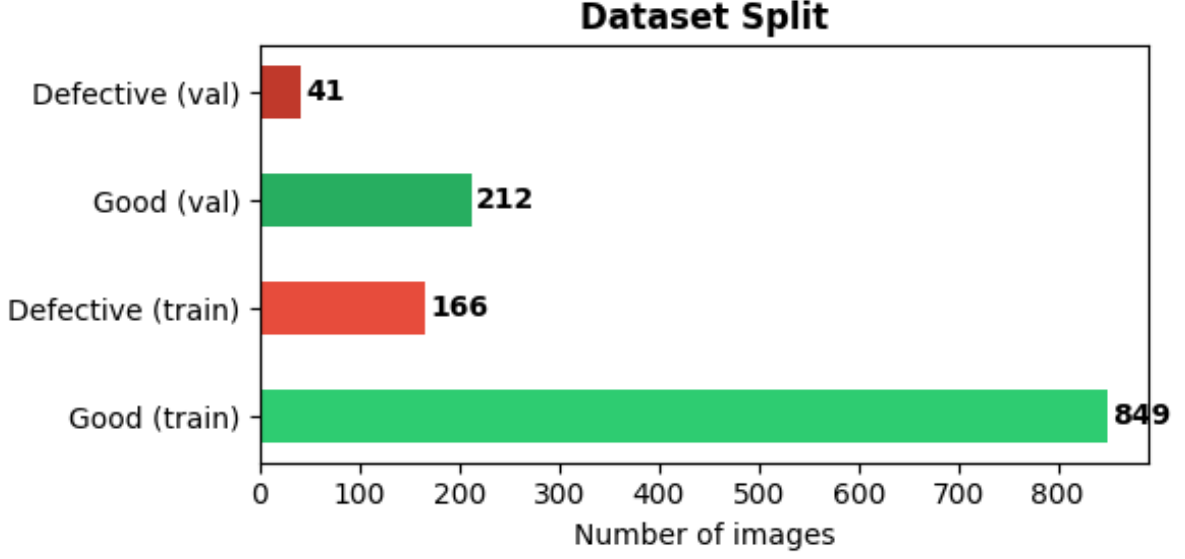


Figure 11: Dataset Split for Binary Classification

5.4 Handling Class Imbalance

The 5:1 class imbalance is addressed through three simultaneous strategies:

1. WeightedRandomSampler: Each training sample is assigned a weight inversely proportional to its class count:

$$w_i = \frac{1}{N_{c(i)}} \quad (13)$$

where $N_{c(i)}$ is the number of samples in the class of sample i . The sampler draws with replacement such that each class is seen approximately equally often per epoch, making the effective training distribution approximately 50/50.

2. Class-weighted loss: The CrossEntropyLoss is weighted by class frequency:

$$\mathcal{L} = - \sum_c w_c \cdot y_c \cdot \log(\hat{y}_c), \quad w_c = \frac{1}{N_c} \quad (14)$$

This further penalises misclassification of the minority (defective) class.

3. F1-based model saving: The best model checkpoint is saved based on the *defective class F1 score* on the validation set, rather than overall accuracy:

$$F1_{\text{defect}} = \frac{2 \cdot P_{\text{defect}} \cdot R_{\text{defect}}}{P_{\text{defect}} + R_{\text{defect}}} \quad (15)$$

This ensures the saved model prioritises defect detection over overall classification accuracy.

5.5 Training Strategy

Training follows a **two-phase strategy** to prevent catastrophic forgetting of ImageNet features:

Phase 1 — Warmup (epochs 1–5): The EfficientNet-B0 backbone is frozen (`requires_grad = False`). Only the 328,450 head parameters are updated at learning rate $\eta = 10^{-4}$. This allows the randomly initialised head to reach a stable state before the backbone is touched.

Phase 2 — Fine-tuning (epoch 6 onwards): The backbone is unfrozen. The backbone learning rate is set to $\eta_{\text{backbone}} = \eta \times 0.1 = 10^{-5}$, making small adjustments to adapt ImageNet features to PCB texture without destroying them.

The remaining training configuration is as follows:

- **Optimiser:** AdamW, $\eta = 10^{-4}$, `weight_decay`= 10^{-4}
- **Scheduler:** CosineAnnealingLR, $T_{\text{max}} = 30$, learning rate decays smoothly to near zero
- **Epochs:** Up to 30, with early stopping (`patience`= 7) based on validation defect F1
- **Batch size:** 16, `num_workers`= 2



Figure 12: Training Curves

5.6 Results

5.6.1 Key Metrics

Table 6: Summary of key evaluation metrics for binary classification.

Metric	Value
AUC-ROC	0.9790
Overall accuracy	87%
Best val defect F1	0.5725

5.6.2 Classification Report

Table 7: Classification report on the validation set (softmax threshold = 0.5).

Class	Precision	Recall	F1-Score	Support
Defective	0.69	0.96	0.80	77
Good	0.98	0.84	0.91	212

Note: The validation defective support shows 77 rather than 41 due to the `WeightedRandomSampler` with `replacement=True` oversampling the minority class during the evaluation dataloader pass.

Figure 13 shows the confusion matrix for this method.

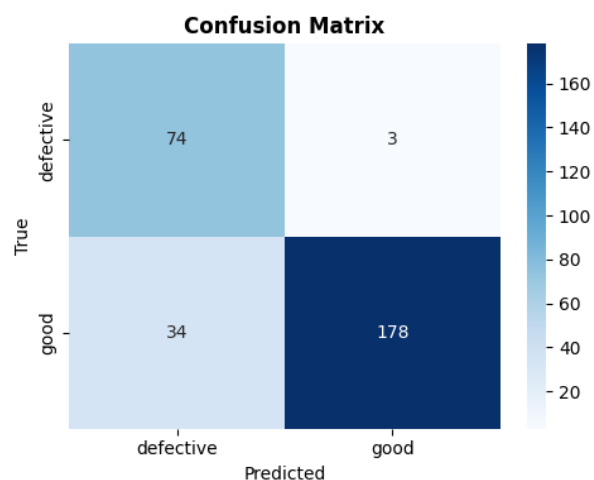


Figure 13: Confusion Matrix

5.6.3 Analysis

The **AUC-ROC of 0.9790** indicates excellent probability-level separation between good and defective patches — the model almost always assigns a higher defective probability to truly defective images than to good ones. The **defective recall of 0.96** confirms that the model catches nearly all real defects in the validation set, which is the critical requirement for a quality control system. The lower defective precision (0.69) reflects a trade-off: some good patches are incorrectly flagged, but this is preferable to missing genuine defects.

The gap between AUC (0.979) and defect F1 (0.5725) arises because AUC measures ranking quality at all thresholds, while F1 is evaluated at a fixed threshold of 0.5. Tuning the decision threshold below 0.5 could improve F1 by further trading precision for recall.

5.7 Advantages and Limitations

Advantages:

- AUC-ROC of 0.9790 demonstrates strong probability-level separation between good and defective patches.
- High defective recall (0.96) — misses very few real defects, critical for quality control applications.
- Three simultaneous imbalance strategies (sampler, weighted loss, F1-based saving) effectively handle the 5:1 class ratio.
- Fast, deployable artefact — single forward pass at inference, model size 17.6 MB.

Limitations:

- Provides no spatial localisation — cannot indicate where in the image the defect is located.
- Cannot distinguish between defect types — output is binary only.
- Best validation defect F1 of 0.5725 is moderate, reflecting the difficulty of the precision/recall trade-off at a fixed threshold.
- Performance is limited by the small defective training set (166 images); adding more defective data would directly improve results.
- Requires retraining if new defect types are introduced.

6 Method 4: YOLOv8 Multi-class Defect Detection

6.1 Motivation

The previous methods either lack spatial localisation or cannot classify defect types. For a production inspection system, the most actionable output is a **bounding box with a class label** — knowing both *where* the defect is and *what kind* it is allows targeted repair decisions. With 105 CVAT-annotated images providing bounding boxes across 15 defect categories, object detection becomes feasible. **YOLOv8s** (small variant) was chosen as the recommended balance between model capacity and dataset size for datasets in the 50–200 image range — large enough to learn meaningful features, small enough to avoid severe overfitting.

6.2 Data Preparation

6.2.1 COCO to YOLO Format Conversion

YOLO requires bounding boxes in normalised centre format. The conversion from COCO $[x, y, w, h]$ (top-left corner, absolute pixels) to YOLO $[c_x, c_y, n_w, n_h]$ (normalised centre) is:

$$c_x = \frac{x + w/2}{W}, \quad c_y = \frac{y + h/2}{H}, \quad n_w = \frac{w}{W}, \quad n_h = \frac{h}{H} \quad (16)$$

where W and H are the image width and height. Boxes with $n_w < 0.001$ or $n_h < 0.001$ are degenerate and discarded. Each image's annotations are saved as a `.txt` file with one row per bounding box: `class_id c_x c_y n_w n_h`. Figure 14 shows some sample annotations of the dataset.



Figure 14: CVAT Annotations

6.2.2 Dataset Split and Augmentation

The 92 images are split **70 / 13 / 9** (train/val/test, approximately 75% / 15% / 10%) with a fixed random seed.



Figure 15: Training Data per-class split

To compensate for the small training set, an aggressive augmentation pipeline is applied using **Albumentations** with a factor of 5 (each training image generates 5 augmented copies), expanding the effective training set from 70 to **420 images**:

- Horizontal and vertical flips ($p = 0.5$ each)
- Random 90° rotation ($p = 0.5$)
- ShiftScaleRotate (shift $\leq 5\%$, scale $\leq 15\%$, rotate ≤ 15 , $p = 0.6$)
- RandomBrightnessContrast (± 0.3 , $p = 0.7$)
- HueSaturationValue ($p = 0.5$)
- GaussianNoise (variance 5–25, $p = 0.4$)
- GaussianBlur (kernel 3–5, $p = 0.3$)
- CLAHE ($p = 0.3$)

Additional YOLO built-in augmentations are applied during training: mosaic ($p = 1.0$), mixup ($p = 0.2$), and copy-paste ($p = 0.3$).

6.3 Model and Training Configuration

6.3.1 YOLOv8s Architecture

YOLOv8s is an anchor-free single-stage object detector with **11,131,776 parameters**, **73 layers**, and **28.5 GFLOPs**. Key architectural features include:

- **Anchor-free detection:** Box dimensions are predicted directly without predefined anchor shapes, making the model more flexible and removing anchor hyperparameter tuning.
- **Decoupled head:** Separate branches for classification and bounding box regression, improving training stability.

- **C2f backbone modules:** Cross-stage partial blocks with two bottlenecks, providing richer gradient flow than standard CSP blocks.
- **Multi-scale FPN neck:** Features extracted at three scales for detection of both large and small defects.

The loss function combines three components:

$$\mathcal{L} = \lambda_{\text{box}}\mathcal{L}_{\text{CIoU}} + \lambda_{\text{cls}}\mathcal{L}_{\text{BCE}} + \lambda_{\text{dfl}}\mathcal{L}_{\text{DFL}} \quad (17)$$

where $\mathcal{L}_{\text{CIoU}}$ is the Complete IoU loss for box regression, $\mathcal{L}_{\text{BCE}}$ is binary cross-entropy per class for classification, and $\mathcal{L}_{\text{DFL}}$ is Distribution Focal Loss for precise box edge localisation.

6.3.2 Training Hyperparameters (Best Run)

Two training runs were conducted. The best results were obtained with the following configuration:

Table 8: Training configuration for the best YOLOv8s run.

Parameter	Value
Framework	Ultralytics 8.4.37, PyTorch 2.10+cu128
Image size	1280×1280
Batch size	4
Epochs	150 (patience = 30)
Optimiser	AdamW
η_0 / η_f	$5 \times 10^{-4} / 0.01$
Weight decay	10^{-4}
Warmup epochs	10
Mosaic / mixup	1.0 / 0.2
Copy-paste	0.3

Figure 16 shows the loss curves and mAP curve for this method.

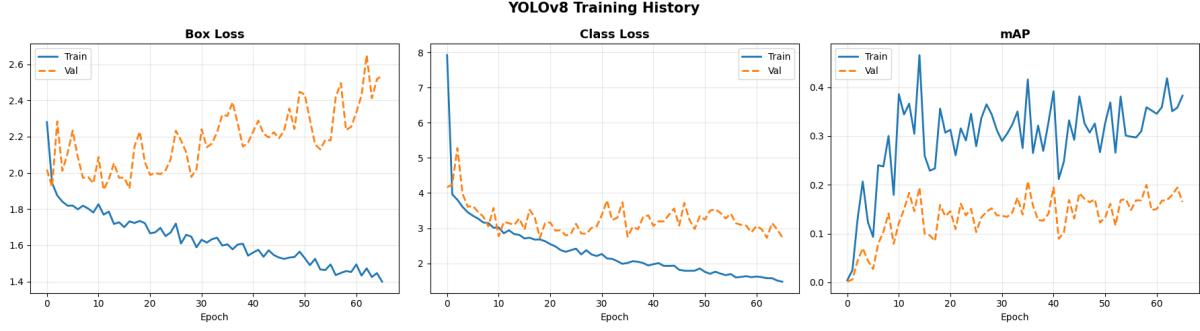


Figure 16: Training Curves

6.4 Explainability: Grad-CAM

Gradient-weighted Class Activation Mapping (Grad-CAM) is applied to visualise which spatial regions drove each classification decision. For a target class c , the Grad-CAM heatmap is:

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right), \quad \alpha_k^c = \frac{1}{Z} \sum_{i,j} \frac{\partial y^c}{\partial A_{ij}^k} \quad (18)$$

where A^k are the activation maps of the target layer (`clf_model.features[-1]`, the last convolutional block), y^c is the class score, and Z is a normalisation constant.

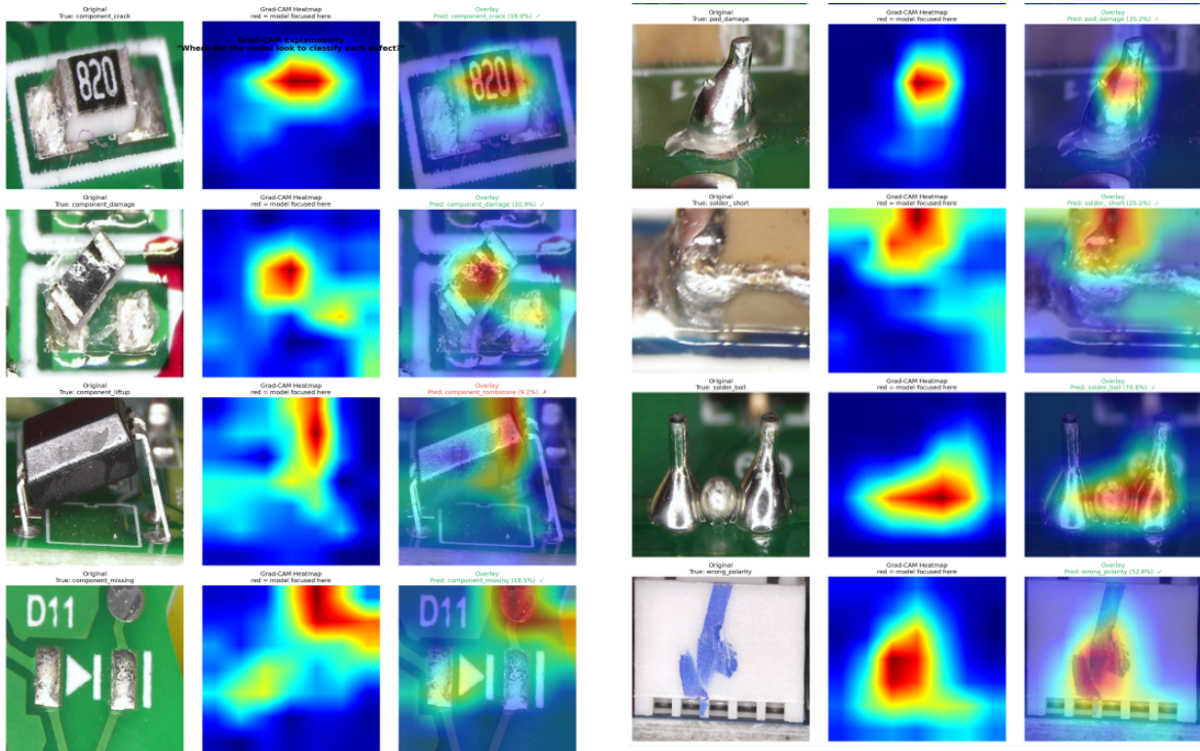


Figure 17: Grad-CAM Explainability of the model

6.5 Results

6.5.1 Test Set Evaluation

Evaluation was performed on the 9-image test split at confidence threshold 0.05 and IoU threshold 0.5.

Table 9: Test set results for the best YOLOv8s run (conf = 0.05, imgsz = 1280).

Metric	Value
mAP@0.5	0.4970
mAP@0.5:0.95	0.2480
Precision	0.4790
Recall	0.5000
Inference speed	45.3 ms/image (Tesla T4)

6.5.2 Per-class AP@0.5

Only 6 of the 15 classes appear in the 9-image test split. Per-class AP@0.5 values are:

Table 10: Per-class AP@0.5 on the test set (conf = 0.05).

Class	AP@0.5
component_crack	0.0000
solder_ball	0.4950
component_no_solder	0.9950
component_damage	0.4950
dry_solder	0.9950
solder_short	0.9950

Figure 18 shows a sample image evaluated from the trained model. The actual defect is component damage which it correctly classifies and locates with 74% confidence, but it also shows component solder short defect with 9% confidence which is incorrect.

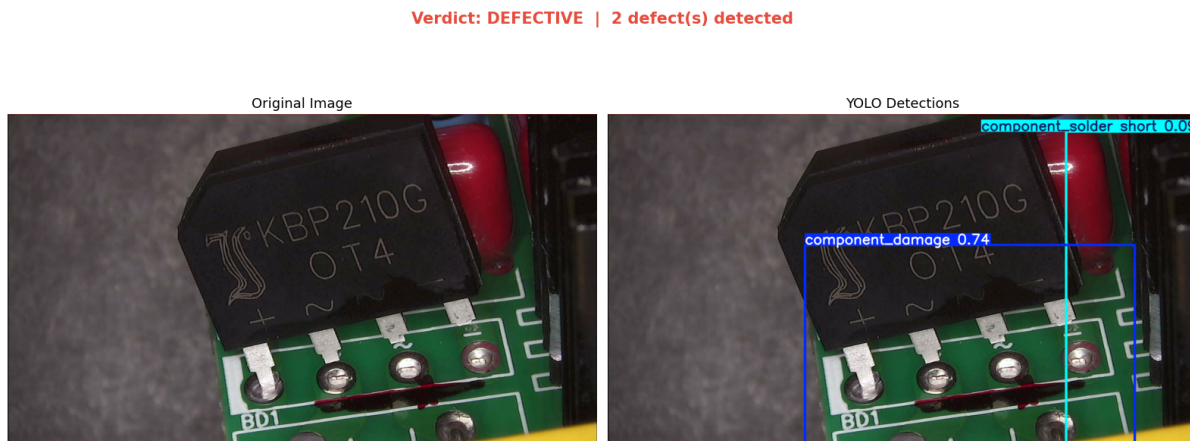


Figure 18: Sample Testing

6.5.3 Effect of Confidence Threshold

The confidence threshold has a strong effect on mAP for this small test set. At `conf= 0.25`, mAP@0.5 drops to 0.249, compared to 0.497 at `conf= 0.05`. Lowering the threshold recovers low-confidence true positive detections that would otherwise be filtered out, which is significant when the test set contains only 10 instances.

6.5.4 Analysis

Classes with more training annotations achieve near-perfect AP (0.995), confirming that the model learns well when sufficient examples are available. Classes with very few annotations achieve AP of 0.0 on the test set, indicating the model failed to generalise for these classes. The primary bottleneck is the dataset size — the augmentation pipeline mitigates this partially but cannot substitute for real annotated examples.

6.6 Advantages and Limitations

Advantages:

- Simultaneously localises and classifies defects in a single forward pass — the most informative output of all four methods.
- Two-stage pipeline (YOLO + crop classifier) refines class predictions on detected regions.
- Grad-CAM provides visual explainability for each classification decision.
- Aggressive augmentation (Albumentations $\times 5$ + YOLO built-ins) substantially expands the effective training set from 70 to 420 images.
- Real-time capable at 45.3 ms per image on Tesla T4 GPU.

Limitations:

- Only 92 usable annotated images — far below what is typically needed for reliable 15-class detection.
- 9 of 15 classes have fewer than 7 annotated instances; the model fails to learn these classes reliably.
- Per-class AP varies widely (0.0 to 0.995), reflecting the highly uneven class distribution.
- The 9-image test set is too small to provide statistically robust evaluation — a single misdetection significantly shifts mAP.
- Confidence threshold selection strongly affects mAP and must be tuned for the deployment use case.
- Requires per-instance bounding-box annotations — the most data-intensive method in this project.

7 Comparative Analysis

This section compares all four methods across key dimensions: output granularity, data requirements, quantitative performance, inference characteristics, and interpretability. The goal is to identify the strengths and appropriate use case for each method, rather than declaring a single winner.

7.1 Output Granularity and Data Requirements

Table 11 summarises what each method produces and what it requires from the dataset.

Table 11: Output granularity and data requirements across all four methods.

Method	Output	Labels quired	Re-	Data Used
Image Subtraction	Binary decision + heatmap	None only)	(pairs	1 good reference per test
Anomaly Detection	Binary decision + anomaly score	None		1061 good patches
Binary Classification	Binary decision + probability	Binary (good/de- fective)		1015 images (80/20 split)
YOLOv8	Bounding box + class label + confidence	Per-instance boxes + class		92 annotated im- ages

Image subtraction and anomaly detection require no defect labels at all, making them immediately applicable even when annotation resources are unavailable. Binary classification requires only image-level binary labels, which are easy to produce. YOLOv8 requires the most effort — per-instance bounding-box annotations with class labels — but produces the most informative output.

7.2 Quantitative Performance

Table 12 compares the key quantitative metrics reported for each method.

Table 12: Key quantitative metrics across all four methods.

Method	Primary Metric	Value	Defect Recall
Image Subtraction	Qualitative (heatmap)	—	Not computed
Anomaly Detection	AUC-ROC	0.8620	0.84 (at $\theta = 0.39$)
Binary Classification	AUC-ROC	0.9790	0.96 (at threshold 0.5)
YOLOv8	mAP@0.5	0.4970	0.50 (conf = 0.05)

Binary classification achieves the highest AUC-ROC (0.9790) and defect recall (0.96), but this metric measures image-level binary discrimination only. Anomaly detection achieves a strong AUC of 0.8620 without using any defect labels. YOLOv8’s mAP of 0.497 is not directly comparable to the AUC scores of the other methods, as it measures localisation quality at a specific IoU threshold in addition to classification correctness — a strictly harder task.

7.3 Capability Comparison

Table 13 provides a structured comparison of capabilities across all four methods.

Table 13: Capability comparison across all four methods.

Criterion	Image Subtraction	Anomaly Detection	Binary Classif.	YOLOv8
Defect localisation	Heatmap	×	×	Bounding box
Defect type output	×	×	Binary	15 classes
Training required	×	×	✓	✓
Defect labels needed	×	×	Binary	CVAT boxes
Handles novel defects	×	✓	Partial	×
Inference speed	Fast	Moderate	Fast	Fast (GPU)
Interpretability	High	Medium	Low	High + Grad-CAM

8 Conclusion and Future Directions

8.1 Conclusion

This project explored four distinct approaches to automated PCB defect detection and classification, each addressing a different point in the trade-off space between label requirements, output granularity, and generalisation capability. The key findings for each method are summarised below.

Image subtraction demonstrated that classical computer vision can effectively detect positional defects — solder spread and component liftup — without any training data or defect labels. The combination of template-based alignment, histogram matching, SSIM scoring, and morphological contour filtering produces interpretable, localised detections. The method is limited to defect types with fixed reference images and requires manual calibration of thresholds and ROI coordinates per PCB variant, making it unsuitable as a general-purpose solution.

Anomaly detection using a WideResNet-50 memory bank and FAISS nearest-neighbour search achieved an AUC-ROC of **0.8620** and a defect recall of **0.84** at the chosen threshold, using no defect labels whatsoever. The separation of 0.2180 between good

and defective score distributions confirms that pretrained ImageNet features provide meaningful discrimination on PCB images without any fine-tuning. The method is the most label-efficient of the four and generalises naturally to unseen defect types, at the cost of being unable to identify what a detected anomaly is.

Binary classification using EfficientNet-B0 achieved the highest AUC-ROC of **0.9790** and a defect recall of **0.96**, demonstrating that transfer learning on a compact architecture can produce strong binary discrimination even with only 207 defective training images. The two-phase training strategy and three-pronged imbalance handling were critical to these results. The method is fast, deployable, and produces calibrated probabilities, but provides no spatial localisation or defect-type output.

YOLOv8s is the only method that simultaneously localises and classifies defects, achieving mAP@0.5 of **0.4970** at the best configuration. Classes with sufficient annotations reached near-perfect AP of 0.995, confirming the architecture is capable. The primary bottleneck is the dataset — only 92 usable annotated images across 16 classes, with 9 classes having fewer than 7 instances. The two-stage pipeline with an EfficientNet-B0 crop classifier and Grad-CAM explainability adds further utility, though the crop classifier accuracy is itself limited by the tiny per-class crop counts.

Overall, the four methods are **complementary rather than competing**. No single method dominates across all criteria. The appropriate choice depends on data availability, annotation resources, and the granularity of output required by the downstream inspection process.

8.2 Future Directions

Three concrete directions are proposed for future work, each directly addressing a bottleneck identified in this project.

8.2.1 Annotate More Images and Increase Dataset Size

The most impactful improvement would be to **collect and annotate more defective PCB images**. A larger and more diverse annotated dataset would directly address the primary bottleneck identified in this project — insufficient per-class training examples. As a first step, the existing 102 unannotated defective images should be labelled with CVAT bounding boxes, which would roughly double the YOLO training set from 70 to approximately 140 images and provide additional instances for the 9 classes currently below the 7-instance threshold. Beyond this, actively collecting new defective samples — particularly for underrepresented classes — would further improve per-class AP and make the model more robust across all defect categories.

8.2.2 Hybrid Model Pipelines

Two hybrid pipeline architectures are proposed, combining the strengths of multiple methods:

Binary + YOLOv8: The EfficientNet-B0 binary classifier acts as a fast first-pass gate. Images with defective probability below a threshold are immediately passed as good, avoiding the computational cost of running YOLOv8 on the majority of good images. Only images flagged as defective by the binary classifier are passed to YOLOv8 for precise bounding-box localisation and class identification. This reduces overall inference cost while retaining the full output granularity of YOLO for flagged images.

Anomaly Detection + YOLOv8: The WideResNet-50 memory bank acts as a broad-coverage anomaly detector, flagging any image that deviates from the normal distribution — including novel defect types not present in the YOLO training set. Images flagged as anomalous are then passed to YOLOv8 for localisation and classification of known defect categories. This hybrid approach reduces false negatives by catching defect types that YOLO was never trained on, while retaining YOLO’s precise, actionable output for known classes.

8.2.3 Semi-supervised Learning with YOLO Pseudo-labels

The 102 unannotated defective images can be leveraged without manual annotation effort through **pseudo-labelling**. The trained YOLOv8 model is applied to these images at a high confidence threshold (e.g. $\text{conf} > 0.70$) to automatically generate bounding-box predictions. High-confidence detections are accepted as pseudo ground truth and added to the training set alongside the 92 manually annotated images. The model is then retrained on this expanded dataset, and the cycle is repeated iteratively — each iteration producing a model capable of generating more reliable pseudo-labels. This bootstraps annotation coverage without additional manual labelling effort and is particularly valuable for classes where the current model already performs well (component crack, component liftup), as it can reliably pseudo-label additional instances of these classes to further improve their representation.